5G RAN Network-Slicing HRL Simulator

Technical Reference: Environment Physics, Upper & Lower RL Agents, and Rule-Based Baselines

Repository: `/home/oussama/Desktop/chech`

Prepared: 10 July 2026

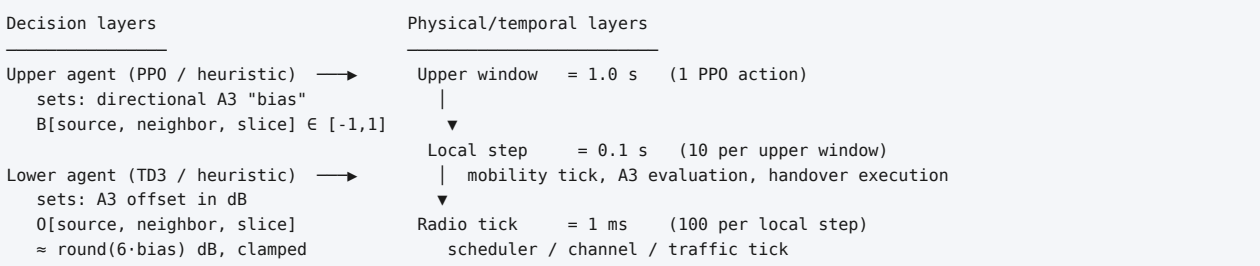
Scope: A code-grounded walkthrough of the `MultiGNBWrapper` / `GlobalPPO3GNBEnv` simulation stack, the upper PPO load-balancing agent, the lower TD3/directional agent, and the deterministic heuristic controllers used as baselines.

Method note: every formula and constant below was extracted directly from the current working-tree source (not from prior design docs) and cross-checked against the relevant unit tests. Where the code has drifted from an earlier design document (`PROJECT_SUMMARY.md`, `hrl_v15_directional_admission.pdf`), the discrepancy is called out explicitly rather than silently reconciled.

Contents

1. System Overview

The repository implements a hierarchical reinforcement-learning (HRL) testbed for inter-cell load balancing in a 3-gNB 5G RAN, using 3GPP A3 handover events as the mechanism of control. The system is organized in three physical/temporal layers and two decision layers:



A deterministic **bias → offset** translation law and a deterministic **safe-admission quota** layer sit between the two decision layers and the physical simulator, so that a learned upper policy, a learned lower policy, and rule-based heuristics can all be evaluated through the identical execution path. This report documents each of those pieces in turn: the environment (Section 2), the upper PPO agent (Section 3), the lower TD3 agent (Section 4), and the rule-based heuristic stand-ins for both (Section 5).

> **Headline finding used throughout this document:** the repository contains an older internal document (`PROJECT_SUMMARY.md`) and an earlier design PDF (`hrl_v15_directional_admission.pdf`) describing safety vetoes and asymmetric offset scaling that are **no longer present** in the code that actually executes. Section 6 consolidates every such correction. Treat formulas in Sections 2–5 as the ground truth of the current working tree.

2. Simulation Environment

File / class map

Layer	File	Class
Upper/global Gym env (PPO)	`global_ppo_3gnb_env.py`	`GlobalPPO3GNBEnv`
World/physics wrapper (mobility, radio, traffic, A3, admission)	`multi_gnb_wrapper.py`	`MultiGNBWrapper`
Cell / base station	`node_b.py`	`NodeB`
Per-slice L1 multiplexer	`slice_l1.py`	`SliceL1eMBB`, `SliceL1URLLC`
UE / packet model	`slice_ran.py`	`UE`, `Packet`
PRB scheduler	`schedulers.py`	`ProportionalFair`
MCS/codeset, legacy propagation	`channel_models.py`	`MCSCodeset`, `SINRSelectiveFading`
Admission budget layer	`safe_admission_controller.py`	`DirectionalAdmissionBudgetController` (alias `SafeAdmissionController`)
A3→offset translation ("lower control law")	`strong_heuristic_local_executor.py`	`strong_directional_heuristic_local_executor`
Traffic sources	`traffic_generators.py`	`CbrSource`, `FixedPacketCbrSource`, `VbrSource`
Scenario/topology factory	`scenario_creator.py`	`create_nodeb`, `create_multignb_env`

2.1 Topology & Entities

Default 3-gNB topology (`DEFAULT_GNB_CONFIGS_3`, used when `scenario_mode="snapshot"`):

```
gNB 0: (x=0,   y=0),   coverage_radius=520 m, carrier_id=0, n_prbs=100
gNB 1: (x=450, y=0),   coverage_radius=520 m, carrier_id=0, n_prbs=100
gNB 2: (x=225, y=390), coverage_radius=520 m, carrier_id=0, n_prbs=100
```

Carrier configuration is scenario-mode-dependent. The default `snapshot` topology puts all 3 gNBs on `carrier_id=0` — i.e. real co-channel interference is modeled between them (§2.3). The `curriculum` topology (the one actually used for upper-agent and lower-agent training, via `center_left_right_gnb_configs()` in `upper_agent_training_scenarios.py`) explicitly sets `carrier_id = gnb_id`, i.e. **no inter-cell interference** is modeled in the topology actually used for training. This is a deliberate simplification for training but worth stating explicitly when interpreting results.

Radio parameters per gNB: `center_frequency_hz=3.5e9`, `bandwidth_hz=20e6`, `tx_power_dbm=30.0`, `noise_figure_db=7.0`, `shadowing_std_db=0.0` (disabled by default; frequency-selective fading is trace-driven instead, §2.3.6), `n_prbs=100`. Coverage is a hexagon inscribed in `coverage_radius` (ray-casting `is_point_in_coverage`), not a plain circle.

UEs

Each UE (`slice_ran.py`, class `UE`) carries position/velocity, `slice_type` (eMBB/URLLC/mMTC), a FIFO packet queue, radio state (SNR/SINR/MCS/PRBs), and cumulative SLA counters. Per-packet SLA deadlines are hard-coded:

Slice	Deadline
URLLC	10 ms
mMTC	1.0 s
eMBB	none — judged on window-average delivered/offered bits ratio instead

Slices & traffic

In the mobile multi-gNB path, PRBs are **not** partitioned per slice: a single gNB-wide Proportional-Fair scheduler allocates the full `n_prbs` pool across all UEs at that gNB regardless of slice; per-slice load is then measured as that slice's *share* of the gNB's useful PRBs. Default CBR traffic profiles (`DEFAULT_UE_TRAFFIC_PROFILES`):

Slice	packet_size_bits	bit_rate
eMBB	3000	200,000 bps
URLLC	800	80,000 bps
mMTC	400	20,000 bps

Traffic is deterministic fixed-size-packet CBR (credit accumulates at `bit_rate × step_length`, emits whole packets), not Poisson.

2.2 Timing / Clock Hierarchy

A strict 3-level clock is enforced by an assertion in `GlobalPPO3GNBEnv.__init__`:

```
radio_substeps * radio_tick_seconds == upper_window_seconds / local_steps_per_global
```

Level	Default	Meaning
Upper window	1.0 s	one PPO / upper decision
Local step	0.1 s	10 per upper window — mobility tick, A3 evaluation
Radio tick	1 ms	100 per local step — scheduler / channel / traffic tick

Within one local step, the full channel + PF-scheduler recomputation runs only every `min(5, radio_substeps)=5` radio ticks (5 ms); the other ticks replay the cached PRB allocation and re-run each UE's `transmission_step()` against the current queue — this gives URLLC's 10 ms deadline two scheduling rounds per window.

Anatomy of one upper-window `step()`

```
GlobalPPO3GNBEnv.step(action):
  B = reshape(clip(action,-1,1), (n_gnbs, max_neighbors, n_slices))     # directional bias tensor
  offsets = strong_directional_heuristic_local_executor(B, ...)          # ONE batched call (§2.6, §5.2)
  apply offsets to A3 thresholds
  begin_safe_admission_window(B)        # freeze per-direction handover quotas (§2.5)
  begin_sla_window()

  ── SETTLE phase (effective_settle_steps local steps, default 4) ──
     each tick: mobility + radio + A3 evaluation + admission-gated handover execution
     mid-window: any direction whose quota is exhausted has its offset neutralized to 0 dB

  begin_radio_measurement_window()      # clean measurement starts only now

  ── MEASUREMENT phase (local_steps_per_global - settle_steps ticks) ──
     same per-tick mechanics, contributes to end_loads / reward

  end_loads   = window-average load (measurement phase only)
  start_loads = previous window's clean measurement
  reward      = f(start_loads, end_loads, ...)   (§2.9)
```

`effective_settle_steps = min(max(post_handover_settle_steps, required_settle_steps), local_steps_per_global-1)`, where `required_settle_steps` guarantees the A3 TTT (3 ticks) plus enough execution ticks for the expected handover quota. Default `post_handover_settle_steps=4`.

Warm-up: `warmup_steps=2` full upper windows are run with zero bias before the first observed episode step, so the SLA/PRB measurement window is populated from a non-cold state; warm-up rewards/handovers are discarded.

Optional dynamic window: if `dynamic_upper_window=True`, `local_steps_per_global` can stretch up to `max_dynamic_local_steps_per_global` (12, or 40 in the baseline-evaluation harness) so a window with a large accepted handover budget gets enough ticks to execute it.

2.3 Channel Model — Pathloss, SINR/RSRP, Interference

Two propagation models coexist in the codebase: the legacy single-cell `channel_models.py` functions (unused by the mobile multi-gNB path) and the `node_b.py` / `multi_gnb_wrapper.py` model actually used by `MultiGNBWrapper`, documented below.

2.3.1 Pathloss — `NodeB.get_received_power_dbm`

```
d_m   = max(distance(gNB, UE), 10.0)
d_km  = d_m / 1000
f_ghz = center_frequency_hz / 1e9

PL_dB = 128.1 + 37.6*log10(d_km) + 20*log10(f_ghz / 2.0)     # urban-macro (3GPP TR36.942-style), 3.5 GHz correction
PL_dB = max(PL_dB + shadowing_dB, 70.0)                        # MCL clamp; shadowing ~ N(0, shadowing_std_db), default
OFF

Rx_total_dBm = tx_power_dbm - PL_dB          (-inf if UE outside the hexagonal coverage area)
```

2.3.2 Per-PRB power / RSRP

```
Rx_per_PRB_dBm = Rx_total_dBm - 10*log10(n_prbs)
rx_total_dBm = get_received_power_dbm(gNB, UE) - env_loss_db   # env_loss_db: optional zone penalty, 0 by default
rsrp_dbm := rx_total_dBm     # the quantity used for A3 (§2.6)
```

2.3.3 Noise — `NodeB.get_noise_power_dbm`

```
RB_bandwidth = 180 kHz                                    # 12 subcarriers × 15 kHz
N_dBm = -174 + 10*log10(180000) + noise_figure_db
      = -174 + 52.55 + 7.0  ≈  -114.45 dBm per PRB          (default noise_figure_db=7.0)
```

2.3.4 Interference — `_compute_interference_watts`

Only **same-carrier** neighbors contribute. For each such neighbor *j* in coverage of the UE:

```
I_watts += 10^((neighbor_j.rx_power_per_prb_dbm(UE) - 30)/10) * duty_factor_j
```

`duty_factor_j` is the fraction of neighbor *j*'s PRBs allocated in the previous full-scheduler substep, clipped to [0,1] — interference scales with the neighbor's **actual instantaneous load**, not a fixed worst-case. An idle same-carrier neighbor contributes zero interference (verified by `tests/test_radio_link_budget.py`: forcing a neighbor's duty factor to 1.0 crashes SINR from >18 dB to <1 dB).

2.3.5 SINR / SNR — `_compute_link_metrics`

```
sig_w, noise_w, interf_w = dBm_to_W(...)
snr_db  = rx_dBm - noise_dBm
sinr_lin = sig_w / max(noise_w + interf_w, 1e-15)
sinr_db  = clip(10*log10(sinr_lin), -20, 40)
```

A UE is marked **radio-disconnected** when `sinr_db` is non-finite or ≤ `disconnect_sinr_db` (default −100 dB, effectively only out-of-coverage).

2.3.6 Frequency-selective fading (scheduler-facing only)

For the PRB-level scheduling decision (not A3/RSRP), a per-PRB fading offset is drawn from one of three trace files (`datasets/fading_trace_{EPA_3kmph,ETU_3kmph,EVA_60kmph}.csv`), giving each UE a 100-length SINR vector across PRBs. Each UE is assigned a trace + random starting index at attachment; a ±1 random walk steps the index each call.

2.3.7 Validated example

`tests/test_radio_link_budget.py` checks `18 ≤ SNR ≤ 24 dB` for a UE 135 m from a 200-PRB gNB at 30 dBm tx, 7 dB NF, 3.5 GHz. Manual check: PL≈100.3 dB → Rx_total≈−70.3 dBm → Rx/PRB≈−93.3 dBm → N≈−114.45 dBm ⇒ SNR≈21.2 dB. ✓ consistent.

2.4 MCS, PRB Scheduling & the Load Formula

MCS codeset

Loaded from `datasets/mcs_codeset.csv`. `nominal_rate(mcs) = rate[mcs] × order[mcs]` bits/symbol; highest entry `rate=0.9, order=6` (64-QAM) ⇒ `nominal_rate_max = 5.4` bits/symbol. A separate, stricter codeset (`mcs_codeset_urllc.csv`) is used for URLLC UEs.

Proportional-Fair scheduler

Called once per gNB per scheduling tick (every 5 radio ticks) over all attached UEs regardless of slice. `granularity=2` PRBs/round, `sym_per_prb=158`, EWMA throughput window `pf_averaging_window_s=0.25 s`. Per resource-block group, the UE with the highest PF score wins:

```
score_i = rate_i * 1[queue_i > 0] / max(th_i, 1)     # rate_i = sym_per_prb * bits_per_symbol_i (from wideband SNR)
```

After coarse allocation, MCS is **re-selected per UE from the actual per-PRB fading vector**:

```
realized_bits  = min(prbs_i * realized_rate, original_queue_bits)
useful_prbs_i  = min(prbs_i, ceil(realized_bits / realized_rate))    if realized_bits>0
wasted_prbs_i  = max(prbs_i - useful_prbs_i, 0)
```

With `sym_per_prb=158` and `nominal_rate_max=5.4`: `bits_per_prb_max = 158×5.4 ≈ 853.2 bits/PRB` at high SINR.

Load formula — three related quantities

The codebase computes load three different ways for three different purposes; conflating them is the single easiest way to misread a training curve.

Quantity	Formula	Used for
Instantaneous used-PRB load	`U = useful_allocated_PRBs / n_prbs`, clipped [0,1]	`estimate_slice_load()` — diagnostic snapshot
Window-averaged useful load	`L = Σ_ticks useful_prbs / (n_ticks × n_prbs)`, clipped [0,1]	`get_window_average_slice_loads()` — saturation counting, some diagnostics
Persistent demand load	`demand_load = Σ_UE upper_demand_prbs / n_prbs` — **not** clipped, can exceed 1 under overload	**What the upper agent actually observes and is scored on** (`_load_matrix()`)

> **Key modeling detail:** the "load" the upper agent sees in its observation and in most reward terms is a persistent, per-UE *offered-demand* quantity, kept conserved across handovers so moving a UE moves its demand contribution atomically — distinct from the window-averaged *useful/served* PRB load used for saturation counting. When reading results, "the agent reduced load imbalance" means *demand* imbalance, not necessarily *served-traffic* imbalance.

2.5 Admission Control

Two independent layers exist; the once-documented single `l_safe` veto is **not** how safety is enforced today.

2.5.1 `DirectionalAdmissionBudgetController` — a pure quota layer

Its own docstring states it explicitly: it does **not** veto on traffic-safety signals (source excess, target load, SLA) — those fields are legacy diagnostics only. Per upper window, for every `(source, target, slice)`:

```
value    = clip(directional_bias[source, target, slice], -1, 1)
strength = max(0, -value)                                  # only negative (release) bias creates quota
direction_budget = 0                                           if strength <= bias_deadband (0.05)
                 = min(ceil(strength * source_ues - 1e-6), source_ues)   otherwise
```

At execution time, candidates are ranked by `(a3_margin ↓, pingpong_ratio ↑, handover_failure_ratio ↑)` and greedily accepted up to `min(step_limit, remaining_episode_budget, direction_quota_remaining)`.

2.5.2 The `l_safe` threshold — computed, but not enforced as a veto

The concrete load-safety threshold (`l_safe = 0.80` default) is a parameter of the **offset-selection law** (`strong_directional_heuristic_local_executor`, §2.6/§5.2), not of the admission controller. Inside that function, `target_is_safe`, `load_safety_db`, and `mobility_safety_db` are computed **and placed only in a debug dict** — they are never subtracted from or used to clamp the applied offset in the code path actually wired into `GlobalPPO3GNBEnv`.

2.5.3 Per-UE / per-episode stability guard (independent of both of the above)

- `episode_budget`: reject once `episode_handover_count ≥ max_handovers_per_episode` (default 20)
- `ue_episode_budget`: reject once a UE has used `max_handovers_per_ue_episode` (default 2)
- `pingpong_guard`: reject an immediate return to the UE's previous serving cell within `a3_pingpong_guard_s` (default 30 s) **unless** serving SINR has dropped below `a3_emergency_sinr_db` (default −5.0 dB), in which case the ping-pong is allowed as an emergency escape

2.6 A3 Handover Mechanism

Trigger condition

```
threshold = RSRP_serving + offset_db + hysteresis_db          (a3_hysteresis_db default 1.0 dB)
trigger:    RSRP_neighbor > threshold
```

Standard 3GPP-A3 form, per-`(serving, neighbor, slice)` offset.

Time-to-Trigger (TTT)

A persistent tick counter per `(gNB, ue, neighbor)` increments once per **local step** (0.1 s) while the condition holds, resets to 0 if it breaks. A candidate matures once the counter reaches `handover_ttt` (default **3** local-step ticks = 0.3 s sustained).

Cooldown & minimum residence

```
a3_handover_cooldown_s = 2.0 s   → 20 local-step ticks   (GlobalPPO3GNBEnv default)
a3_min_residence_s     = 2.0 s   → 20 local-step ticks
if time_since_handover < cooldown:                          skip A3 entirely
elif time_since_handover < min_residence and serving_SINR > emergency_threshold:  skip
else: evaluate normally (emergency escape allowed even inside min-residence)
```

This state persists across upper-window boundaries (cleared only on episode reset).

Bias → A3-offset translation law (§5.2 has the full derivation)

```
bias_deadband = 0.05
|bias| ≤ 0.05:  raw_offset = 0                       (neutral)
bias > 0.05:    raw_offset = 6.0 * bias               (retain, positive dB → harder to leave)
bias < -0.05:   raw_offset = 6.0 * bias               (release, negative dB → easier to leave)
proto_offset  = clip(raw_offset, -12.0, +6.0)
applied_offset = snap to nearest 1 dB grid point in {-12,...,-1,0,1,...,6}
```

i.e. `offset_db ≈ round(6·bias)` dB, clamped to `[−12, +6]` dB — asymmetric because the release side needs enough headroom to force a handover even against a positive RSRP margin, while the retain side only needs to reach +6 dB to fully block A3 under realistic margins.

2.7 Observation Space (Upper Agent)

```
obs_dim = n_gnbs*n_slices*3 + n_gnbs + action_dim = 3*3*3 + 3 + 18 = 48
```

Block	Size	Content
`loads`	9	persistent demand load per (gNB, slice), **unclipped** (§2.4)
`counts`	9	connected UE count per (gNB, slice), normalized by `K_max(slice)`
`sla`	9	SLA severity per (gNB, slice), clipped [0,1]
`gnb_demand_load`	3	per-gNB total demand load (sum over slices) — exactly the quantity the default reward optimizes
previous bias	18	last applied directional bias tensor (the agent's own last action)

Non-finite values sanitized via `nan_to_num(nan=0, posinf=1, neginf=-1)`. A differently-shaped `MultiGNBWrapper.get_global_agent_observation()` exists but is **dead code** — not used by `GlobalPPO3GNBEnv`.

2.8 Action Space (Upper Agent)

```
action_dim = n_gnbs * max_neighbors * n_slices = 3 * 2 * 3 = 18
action_space = Box(-1.0, 1.0, shape=(18,))
```

One continuous scalar `B[source, neighbor_slot, slice] ∈ [-1,1]` per directed `(source cell → neighbor cell, slice)`. With 3 gNBs each having exactly 2 neighbors, this reshapes to `(3, 2, 3)`. Negative `B` = release pressure toward that neighbor/slice; positive = retain pressure. The tensor also directly sets the admission-quota strength for that direction (§2.5.1) and is held constant for the whole upper window.

2.9 Reward Function

Many shaping terms are computed and logged every step, but under the **default configuration (`upper_reward_mode = "paper_cost"`)** only a specific subset actually drives the gradient. This is the single most important subtlety in the reward design.

Active by default — `paper_cost_reward`

```
paper_cost_reward = -(
      2.0  * std(per-gNB total demand load)                    # global_reward_mu
    + 3.0  * mean(max(demand_load - effective_load_target, 0))  # paper_excess_load_penalty_weight
    + 0.0  * paper_handover_ratio                               # off by default
    + 0.0  * paper_pingpong_ratio                               # off by default
    + 0.05 * contradictory_bias_penalty_raw                     # mutual-offload contradiction
    + 0.0  * idle_slice_bias_penalty_raw                        # off by default
    + 0.4  * sinr_deficit_penalty_raw                           # SINR-floor deficit
)
reward = paper_cost_reward + expert_bias_bonus                  # (always added, see below)
```

`effective_load_target = max(gnb_load_target=0.65, total_demand/total_capacity)` — the 0.65 target is relaxed upward if the whole network is legitimately more loaded than that, so an infeasible scenario doesn't manufacture unavoidable penalty. `contradictory_bias_penalty` sums `min(|forward|,|reverse|)²` over mutual negative-bias pairs (both sides trying to offload onto each other for the same slice). `sinr_penalty` uses a 5.0 dB floor, averaged only over cells with active demand at window end.

Expert-bias guidance bonus (always active, weight 0.5 by default)

```
expert = precomputed heuristic bias tensor for the current scenario (from a CSV baseline run)
mask   = |expert| > deadband
closeness = clip(1 - rmse(bias[mask]-expert[mask]) / rmse(expert[mask]), -1, 1)
expert_bias_bonus = expert_bias_reward_weight * closeness      # +1 = exact match, 0 = zero action, -1 = opposite
```

This gives the agent up to ±0.5 reward purely for how closely its bias matches a rule-based baseline for the current scenario — a meaningful fraction of typical `paper_cost_reward` magnitude, and active regardless of reward mode.

Terminal-reward gating

```
terminal_reward_only = True (default)
reward = episode_terminal_reward() if truncated else 0.0
```

With `global_steps_per_episode=12`, **11 of every 12 transitions return reward 0**, and only the final step's window reward is nonzero by default — see Section 3 for why this matters more than it sounds (episodes in the actual training scenario catalog are 1 step long, making this less consequential than the name suggests).

Computed but diagnostic-only under the default mode

These are logged into `info[...]` every step but **not summed** into the default `paper_cost_reward`: `load_reward` (normalized PRB-balance improvement), `saturation_reward`, `excess_load_reward` (an alternate excess-load formula), `action_penalty` (bias-smoothness), `negative_bias_penalty`, `served_share_reward` (explicitly zeroed to avoid double-counting), `sla_reward` (hardcoded 0.0 — "deliberately excluded from the upper routing objective"), `jain_reward`. Two alternate full reward modes (`paper_cost_delta`, `paper_cost_target_delta`) exist and use different formulas entirely (improvement-based, target-aware) — see the module for exact terms if a non-default run is being analyzed.

Constant	Default	Role
`global_reward_mu`	2.0	weight on cross-gNB demand-load std
`paper_excess_load_penalty_weight`	3.0	weight on mean excess-over-target demand load
`contradictory_bias_penalty_weight`	0.05	mutual-offload contradiction penalty
`sinr_penalty_weight` / `sinr_floor_db`	0.4 / 5.0	SINR-floor deficit penalty
`gnb_load_target`	0.65	nominal per-gNB total-load target
`expert_bias_reward_weight`	0.5	expert-imitation bonus
`terminal_reward_only`	True	only last-window reward nonzero
`global_steps_per_episode`	12	upper windows per episode

3. Upper Agent — Global PPO Load Balancer

Trained by `train_upper_ppo_3gnb.py` against `GlobalPPO3GNBEnv` (Section 2). Action/observation spaces and reward are defined by the environment itself (§2.7–2.9); this section covers the algorithm configuration, curriculum, and evaluation methodology specific to the training script.

3.1 PPO Configuration

```
PPO("MlpPolicy", env,
    learning_rate = args.learning_rate,   # default 1e-4
    gamma         = 0.99,                  # hard-coded, not a CLI flag
    gae_lambda    = 0.95,                  # hard-coded
    clip_range    = 0.2,                   # hard-coded
    n_steps       = args.ppo_n_steps,      # default 4096
    batch_size    = args.ppo_batch_size,   # default 256
    n_epochs      = args.ppo_n_epochs,     # default 5
    ent_coef      = args.ent_coef,         # default 0.0
    policy_kwargs = dict(log_std_init=args.log_std_init),  # default -1.0
    seed          = args.seed,             # default 7
)
```

Network: SB3 default `MlpPolicy` architecture (two 64-unit tanh layers, split `pi=[64,64]` / `vf=[64,64]`) — `net_arch` itself is never overridden, only `log_std_init=-1.0` is set (std≈0.368 instead of SB3's default 1.0), explicitly to reduce early boundary-clipping against the `[-1,1]` action box.

> **Structural caveat — 1-step episodes.** Every scenario in the current training catalog has `duration_s=1.0` and the default `upper_window_seconds=1.0`, so `episode_steps = 1`: every rollout is `reset()` immediately followed by exactly one `step()` with `truncated=True`. This is confirmed by the shipped training log (11,711 rows → 11,710 episodes, a 1:1 ratio) and by `tests/test_upper_one_step_directional_episode.py`. Practically, upper-agent training is a **contextual-bandit-style PPO** over the scenario pool, not a multi-step trajectory optimization — GAE bootstrapping across steps within an episode never happens, and the bias-smoothness penalty (§2.9) is explicitly zeroed whenever episode length ≤ 1.

3.2 Training Curriculum / Scenario Variety

Scenario catalog (`upper_agent_training_scenarios.py`) — **11 fixed scenarios**, all built on a stationary, symmetric collinear 3-gNB topology (left/center/right, ±X m from center, 6 UEs, `speed_mps=0.0`):

#	Scenario	Slices	Notes
1	`jain_balance_controllable`	eMBB	canonical: 0 dB RSRP tie at ±135 m, bias alone controls A3
2–3	`jain_control_{urllc,mmtc}`	single slice	same geometry, other slices
4	`jain_control_mixed`	all 3	tests all action dims must open together
5–7	`jain_control_{embb_urllc, embb_mmtc, urllc_mmtc}`	2-slice combos	total demand above safe target
8–10	`jain_control_outer_congested[...]`	1–2 slices	outer gNBs congested, center empty — tests inward handover
11	`jain_control_embb_urllc_impossible`	2-slice	**deliberately infeasible** — no light target exists; tests the policy learns not to force useless handovers

3 topology variants (gap only): `tight_220m`, `medium_270m` (default), `wide_320m`. Each gNB gets its own carrier (no inter-cell interference, §2.1).

Selection modes (`--scenario-selection`): `cycle` (default — deterministic round-robin over the pool), `random`, `staged` (tiered progression, degenerates to cycling since all scenarios are currently `tier="fixed"`), `block` (repeat one scenario for N episodes, auto-sized to guarantee a target number of PPO updates per scenario). Two hard-coded phased curricula also exist: `controllable_type1_then_mixed` (single-slice scenarios → mixed, 15k+30k timesteps) and `controlled_slice_curriculum` (single→two-slice→mixed, 5k+10k+20k timesteps).

3.3 Expert-Bias Guidance ("curriculum trick")

Not imitation pretraining — a small potential-based shaping bonus layered on top of whichever reward mode is active (formula in §2.9). The "expert" bias tensor comes from `compute_upper_expert_directional_bias` (the same rule-based heuristic described in §4.6/§5.1), pre-computed offline per scenario into `results/upper_heuristic_3gnb_baseline/upper_heuristic_3gnb_scenario_summary.csv` and loaded at env construction. No annealing schedule exists — the weight is constant throughout training (CLI default 0.5; the shipped run used 0.05, a 10× weaker pull).

> The CLI exposes `--expert-bias-closeness-threshold` (default 0.95) but this value is only logged, never read inside `_expert_bias_guidance_reward` — there is no gating by the threshold despite what the help text implies.

3.4 Evaluation Methodology

A separate `Monitor(GlobalPPO3GNBEnv)` instance evaluates the policy after every completed PPO rollout (`--eval-interval-updates=1`), `deterministic=True` , over `--eval-episodes=10` episodes. Composite score:

```
score = mean_eval_return + qos_weight * mean_network_delivery_ratio - stability_weight * mean_pingpong_ratio   (weights default 1.0/1.0)
```

Whenever score improves, `upper_ppo_best.zip` is saved immediately; `--eval-patience=10` non-improving evaluations triggers early stopping (0 disables). At the end of training, final model selection **prefers `upper_ppo_best.zip` over the raw end-of-training weights** ("training can drift past its peak"), and a final 10-episode deterministic evaluation writes `validation_log.csv` .

3.5 A Real Training Run (`models/upper_ppo_3gnb/run_20260701_143712`)

Largest/most recent run on disk (11,711 logged steps). Notable deviations from current CLI defaults:

`max_handovers_per_local_step`	3 (CLI default 1)
`paper_handover_penalty_weight`	1.0 (CLI default 0.0 — this run actively penalizes handover volume)
`paper_pingpong_penalty_weight`	5.0 (CLI default 0.0)
`contradictory_bias_penalty_weight`	1.0 (CLI default 0.05, 20× stronger)
`expert_bias_reward_weight`	0.05 (CLI default 0.5, 10× weaker)
`ppo_n_steps / ppo_batch_size`	512 / 128 (CLI defaults 4096 / 256)

This run's `config.json` predates several newer script fields (`ent_coef` , `log_std_init` , eval-patience knobs are absent) — it was produced by an earlier revision of the training script.

Tail-1000-row statistics from this run: `mean(reward) ≈ -1.183` , `mean(paper_cost_reward) ≈ -1.224` , `mean(handover_count) ≈ 4.16` /window, `mean(expert_bias_reward) ≈ 0.041` , `mean(network_delivery_ratio) ≈ 0.568` . Only 8 of the current 11 scenarios appear in its cycling order — the 3 `jain_control_outer_congested*` scenarios were added after this run launched.

3.6 Related Script: `train_csv_offset_ppo.py`

Not an alternate upper trainer — a **different, lower-level** PPO agent that outputs raw A3 offsets directly for one single controlled gNB, given an upper bias supplied externally from the same expert CSV, replacing the heuristic executor for just that gNB. Its own config differs materially: `learning_rate=3e-4, n_steps=128, batch_size=64, gamma=0.95, gae_lambda=0.90, ent_coef=0.01, net_arch=[128,128]` , genuine multi-step episodes (`episode_steps=40`) — appropriately shaped for a single-gNB, denser-signal problem rather than the upper agent's one-shot directional decision.

4. Lower Agent — Local Directional (TD3) Agent

> **Working-tree caveat:** the TD3 training script (`train_local_a3_td3.py`) and all saved `models/local_a3_*` checkpoints are deleted from the current working tree (staged deletions, per `git status`) but still exist at the last commit. Hyperparameters below were recovered via `git show HEAD:...` . The environment definitions (`local_a3_agent_wrapper.py` , `local_a3_training_env.py`) are present and current, but have been substantially rewritten since that last TD3 checkpoint was produced — the checkpoint would need retraining against the current environment to be valid again.

4.1 What It Controls (Action Space)

`LocalA3OffsetEnv` (`local_a3_agent_wrapper.py`) — **one instance per gNB** (decentralized execution). Action: continuous, directional, slice-specific A3 offsets, one scalar per `(neighbor, slice)` :

```
action_space = Box(-6.0, 6.0, shape=(n_blocks,))     # n_blocks = |neighbors| * |slices| = 2*3 = 6 for the 3-gNB line
topology
```

Raw output is snapped to the discrete set `{-6,-4,-2,0,2,4,6}` dB via `quantize_a3_offset()` — the coarser "early-debugging" set, not the extended `[-12,+6]` 1 dB grid used by the deterministic heuristic executor. Same sign convention as the upper agent: negative = easier handover out (offload), positive = harder (retain).

The lower agent implements its **own TTT counter** (`ttt=1`) rather than the base simulator's `handover_ttt=3` — it fires on the very next qualifying local step, not after 3 base-env ticks.

Rate limiting: both training wrappers hold the raw action for `action_hold_steps` (default 5) local steps and clip the per-decision delta to `max_offset_change_db=2.0` dB before quantizing — the environment executes a rate-limited, held, quantized version of the policy output.

What it does not control: per-UE admission accept/reject and scheduling weights remain with the deterministic `DirectionalAdmissionBudgetController` (§2.5.1) — exactly the v15 design split: A3 offset controls radio *eligibility*; safe admission controls accepted handover *intensity*, and the two are not learned by the same component.

4.2 What It Observes

Per `(neighbor j, slice s)` block, 9 features:

```
[ B_ijk,                # upper bias i→j, slice k (the active control signal)
  B_jik,                # reverse-direction bias j→i (context only)
  local_count / K_ref,    # normalized UE count at serving gNB, slice k
  neighbor_count / K_ref, # normalized UE count at neighbor gNB, slice k
  v_i, v_j,               # SLA severity at serving / neighbor gNB, slice k
  offset / 6.0,            # currently-applied offset, normalized
  HF_ijk, PP_ijk ]         # handover-failure ratio, ping-pong ratio for this direction
```

plus a global context suffix, 2 values per (gNB × slice) across the whole topology: `[load_g, sla_g]` (preferring live demanded-PRB load over served load).

```
obs_dim = n_blocks*9 + n_gnbs*n_slices*2 = 6*9 + 3*3*2 = 54 + 18 = 72
```

> This is a code-level deviation from the design PDF's minimal 10-feature block: the implementation substitutes raw slice-load with normalized *UE counts*, drops the explicit target-headroom and ΔRSRP terms from the per-block vector, and adds a global load/SLA suffix not specified in the design document.

4.3 RL Algorithm — TD3 (recovered from last commit)

```
action_noise = NormalActionNoise(mean=0, sigma=0.25)     # args.action_noise_sigma default 0.25

TD3("MlpPolicy", env,
    seed=7, action_noise=action_noise,
    learning_rate=1e-3, buffer_size=100_000,
    learning_starts=min(500, max(10, timesteps//10)),
    batch_size=128, gamma=0.95,
    train_freq=(1,"step"), gradient_steps=1,
    policy_kwargs={"net_arch":[64,64]})
model.learn(total_timesteps=20_000)     # default; last saved run used 50_000
```

SB3 default twin critics / target networks / policy smoothing (no `policy_delay`, `target_policy_noise`, or `target_noise_clip` overrides).

Last committed evaluation (`models/local_a3_fixed_feasible_mixed/fixed_snapshot_eval.json`, run `20260608_233050`, 50k timesteps, 40 episode steps) shows **imperfect slice isolation**: e.g. for scenario `feasible_embb_offload_only`, mean applied offset −3.625 dB with eMBB correctly negative but URLLC/mMTC directions leaking negative offset too (`passed_by_slice: {eMBB: true, URLLC: true, mMTC: false}` on some entries) — the known open issue as of that checkpoint.

A **behavior-cloning fallback** now also exists (`lower_rl_training_utils.py`, `BCPolicy`: MLP `[128,128]` + tanh, output scaled by 6.0), trained to imitate `compute_expert_action()` — i.e. the deterministic heuristic executor — suggesting the project is supplementing pure TD3 exploration with imitation of the known-correct rule-based policy.

4.4 Reward Function

`LocalA3OffsetEnv._compute_local_reward()` — five additive terms, per gNB per local step:

```
reward = sla_penalty + balance_penalty + mobility_penalty + smoothness_penalty + bias_align_penalty
```

Term	Formula	Weight
SLA	`-w_sla * mean(sla_flags across all gNB,slice)`	1.0
Load-balance	`-w_balance * Σ_slice Var(instantaneous slice load across all gNBs)`	3.0
Mobility	`-(w_hf*hf_rate + w_pp*pp_rate)`, this-step handover stats only	1.0 / 0.5
Smoothness	`-λ * Σ((proto_offset - prev)/12)²`	λ=0.05
Bias-alignment	`-w * max(0, -(bias * applied_offset/6))`, active only when |bias|>0.1	0.3

Slice-specific SLA severity: eMBB uses `max(throughput_gap, queue_overage)`; URLLC uses `clip((max_delay-10ms)/40ms, 0, 1)`; mMTC uses `clip((drop_ratio-1%)/9%, 0, 1)`.

> This weighting materially differs from the design PDF's proposed reward: the implementation drops the explicit target-overload and raw handover-cost terms, replaces a neighborhood-only balance-*improvement* term with a **global, instantaneous** (non-delta) variance penalty, and adds a bias-alignment term not present in the PDF at all. The per-step reward also adds the base environment's own step reward on top of these five terms.

4.5 Directional Admission — the "v15" Design vs. What's Implemented

Ground-truth design doc: `hrl_v15_directional_admission.pdf`. Its core idea: a single per-gNB-per-slice offset either fires too weakly to reach a distant target, or — once strengthened — fires *all* eligible UEs simultaneously ("total migration"). The fix splits the mechanism in two:

```
Global bias b_{i,s}  ⇒  Local directional offset O_{i,j,s}  ⇒  A3 eligibility  ⇒  Safe admission  ⇒  HO execution
```

The PDF's admission-capacity formula (§9.4) is:

```
A_ijs = ceil( P_off_is * min(E_is, H_js) * K_is )
E_is = max(0, L_is - L̄_s)          # source excess over slice-average load
H_js = max(0, L̄_s - L_js)          # target headroom under slice-average load
```

> **Implemented admission (§2.5.1) omits the `min(E,H)` load term entirely:** `direction_budget = ceil(|bias| · source_UEs)`, independent of target load or source excess. Confirmed by the class docstring and by `tests/test_v15_directional_admission.py::test_admission_quota_uses_bias_times_source_ues_and_is_enforced` (bias=-0.8, 10 UEs ⇒ budget=8=ceil(0.8×10), independent of target load). The PDF's hard target-safety constraint (`L_j + ΔL ≤ L_safe`) is likewise not enforced inside the admission controller. This is a genuine simplification relative to the documented v15 spec, not merely a naming discrepancy.

The proportional pressure-splitting across multiple safe neighbors *is* implemented, but at the **offset-selection** stage (`coordinated_neighbor_offsets`, §5.1) rather than at admission — see Section 5 for the exact scoring formula.

4.6 Training Curriculum

Two parallel training environments:

(a) `LocalA3RuleBiasTrainingEnv` — Stage 1, no upper env, no SUMO

Uses `EpisodeTrainingScenario` objects assigning per-slice cases (`offload` / `retain` / `neutral` / `risky_offload`), each forcing deterministic target loads via seeded UE demand/queue. Two named catalogs: `DEFAULT_LOCAL_A3_TRAINING_SCENARIOS` (7 scenarios) and `FEASIBLE_MIXED_SLICE_SCENARIOS` (8 scenarios, behind the last committed checkpoint). A separate 17-case "demand curriculum" generator randomizes gNB×slice demand matrices per named overload pattern.

(b) `UpperScenarioLowerEnv` — current primary trainer

Wraps a live `GlobalPPO3GNBEnv` and reuses the same 9-scenario Jain-fairness catalog as upper-agent training (§3.2). **One controlled gNB** (default gNB-1) is TD3-driven; all other gNBs run the deterministic heuristic executor every step before the controlled gNB's logic runs — genuinely decentralized execution with N−1 heuristic agents and 1 learning agent. Episode structure: 40 steps, 1.0 s upper window, 10 local steps/window, action held 5 steps, offset rate-limited to 2 dB/decision.

> Safe-admission is active by default in `UpperScenarioLowerEnv` (inherited from `GlobalPPO3GNBEnv`) but **inactive by default** in `LocalA3RuleBiasTrainingEnv` (which calls `create_multignb_env` directly and never sets `safe_admission_enabled`) — a meaningful realism gap between the two lower-agent training environments.

4.7 Integration Status: Upper ↔ Lower

> **The hierarchical loop is not currently closed end-to-end.** `GlobalPPO3GNBEnv.lower_agents = {}` is declared and iterated in `reset()` but is **never populated anywhere in the codebase** — confirmed by a repository-wide search. Concretely: the upper PPO agent is trained and evaluated (`train_upper_ppo_3gnb.py` , `run_trained_policy_eval.py`) entirely against the deterministic rule-based heuristic acting as the lower controller (§5.2), never against the TD3 policy. The TD3/BC lower agent is trained and evaluated in isolation (§4.6), and its last checkpoint has since been removed from the repository. If this hierarchical claim is central to the intended contribution, closing this integration gap — wiring a trained lower policy into `GlobalPPO3GNBEnv` 's per-local-step execution — is the largest remaining architectural item.

The one piece of infrastructure genuinely shared across both regimes: both the TD3-controlled gNB during lower-agent training and the heuristic-controlled gNBs during upper-agent training pass through the identical `DirectionalAdmissionBudgetController` quota mechanism, fed by the same bias tensor.

5. Heuristic Baselines — Upper & Lower

Two independent upper-bias generators and two local-executor variants exist in the codebase. Only one pair is actually wired into `run_heuristic_baseline.py` (the fair, apples-to-apples comparison point against the RL agents); the report distinguishes "exists in the file" from "is on the live evaluation path."

Role	Function	File	Actually used by
Upper heuristic #1 (simpler)	`coordinated_neighbor_offsets`	`two_neighbor_offset_heuristic.py`	Only inside `LocalA3OffsetEnv`'s internal "desired offset" reference (§4) — not the live baseline's upper bias source
Upper heuristic #2 (live)	`compute_upper_expert_directional_bias`	`local_a3_training_env.py`	**`run_heuristic_baseline.py`** and the expert-bias CSV that feeds §3.3
Lower executor A (legacy)	`strong_heuristic_local_executor`	`strong_heuristic_local_executor.py`	Only its own unit tests retired
Lower executor B (live)	`strong_directional_heuristic_local_executor`	`strong_heuristic_local_executor.py`	**`GlobalPPO3GNBEnv`** — for both the heuristic baseline and any RL agent evaluation

5.1 Upper Heuristic #1 — `coordinated_neighbor_offsets` (used inside lower-agent training only)

```
coordinated_neighbor_offsets(*, source_id, neighbor_ids, slice_types, directional_bias, useful_load,
                             sla_severity=None, handover_failure_ratio=None, pingpong_ratio=None,
                             best_radio_margin_db=None, safe_load=0.80, one_ue_load=0.15,
                             bias_deadband=0.05, max_target_rsrp_deficit_db=8.0)
```

Per slice: `balance_target = mean(source_load, all neighbor_loads)`; `source_pressure = clip01(max(source_load-balance_target,0) / max(0.80-balance_target, 0.05))`.

- **Positive bias** (>0.05): `offset = clip(6.0*bias, 0, 6)` — independent per neighbor.
- **Neutral** (|bias|≤0.05): `offset = 0`.
- **Negative bias** (<-0.05): neighbors *compete* for a shared release budget rather than each independently getting −6 dB:

```
headroom      = max(0.80 - target_load, 0)          # vetoed to 0 if headroom ≤ 0.25*0.15 = 0.0375
radio_score   = clip01((radio_margin + 8.0) / 8.0)   # vetoed to 0 if target radio-infeasible
risk          = clip01(sla_severity + handover_failure_ratio + pingpong_ratio)
score_j       = |bias| * clip01(headroom/0.80) * radio_score * (1 - risk)
offset_j      = clip(-6.0 * source_pressure * (score_j / Σscore), -6, 0)
```

Verified by test: two equally-loaded, radio-symmetric neighbors under −1.0 bias each get exactly −3.0 dB (50/50 split); a near-saturated neighbor (0.79 load) is vetoed to 0.0 and the other absorbs the full pressure; a positive bias of 0.5 maps to exactly +3.0 dB (no competition on the retain side).

5.2 Live Lower Heuristic — `strong_directional_heuristic_local_executor`

Called **once per upper window** from `GlobalPPO3GNBEnv._compute_strong_local_offsets` — the same execution path regardless of whether the upper bias came from the heuristic (§5.5) or the RL policy, which is what makes it a fair shared baseline layer.

```
strong_directional_heuristic_local_executor(B, prev_offsets, ue_slice, ue_serving_gnb, rsrp_matrix,
    neighbor_graph, load, sla_violation, ho_failure_ratio, pingpong_ratio,
    hysteresis_db=1.0, l_safe=0.80, max_target_rsrp_deficit_db=8.0,
    allow_extended_negative_offsets=True, alpha_load=12.0, alpha_mobility=3.0, eta=0.0)
```

Candidate sets: base `{-6,-4,...,6}` (2 dB), or (production default) `EXTENDED_1DB_OFFSET_SET_DB = {-12,-11,...,-1,0,1,...,6}` (1 dB grid).

Per direction: `radio_feasible`, `target_is_safe`, `load_safety_db = alpha_load * max(target_load-l_safe,0)`, and `mobility_safety_db = alpha_mobility*(ho_failure_ratio+pingpong_ratio)` are all **computed and returned only in a debug dict**. The applied offset formula:

```
raw_offset    = 6.0 * bias                                    (both signs — symmetric)
proto_offset  = clip(raw_offset, -12.0, +6.0)
proto_offset  = (1-eta)*proto_offset + eta*prev_offset         (eta=0.0 by default — disabled)
applied_offset = snap_to_nearest_grid_point(proto_offset)
```

> **No safety veto fires in this function.** Confirmed by `tests/test_v15_directional_admission.py::test_unsafe_target_does_not_rewrite_policy_offset`: even with target_load=0.90 > l_safe=0.80, a source bias of −1.0 still produces exactly −6.0 dB toward that overloaded target. Safety is enforced entirely downstream, by (a) the admission-controller quota (§2.5.1), (b) hard handover-count caps, (c) the pingpong/episode stability guard, and (d) mid-window offset neutralization once a direction's quota is exhausted — not by any veto inside the offset-selection formula itself.

5.3 `kbrl_control.py` — Present but Dormant

A kernel-based online-learning admission controller (`KBRL_Control`, using a budgeted online kernel-Perceptron / "Projectron", `algorithms/projectron.py`) for a **completely different problem formulation**: per-cell PRB budget admission with no notion of gNB mobility, A3, or handovers at all — a leftover from an earlier, simpler single/multi-cell PRB-budgeting project. `create_kbrl_agent()` exists in `scenario_creator.py` but is never called anywhere else in the repository. It should not be presented as a baseline comparable to the upper/lower A3-offset heuristics or the RL agents — it belongs to a retired problem formulation within the same repo.

5.4 Live Upper Heuristic — `compute_upper_expert_directional_bias`

Substantially richer than §5.1: uses PRB-demand load as ground truth and additionally guards against stacking multiple slices onto the same physically-overloaded target. Per-slice parameters:

Slice	deadband	safe_load	urgency
eMBB	0.08	0.80	1.0
URLLC	0.05	0.72	1.3
mMTC	0.10	0.85	0.9

Also tracks a whole-gNB (all-slices-summed) safety threshold `safe_total_gnb_load=0.65`, so a gNB overloaded in aggregate pushes bias even for an individually-fine slice: `source_pressure = clip01(max(slice_pressure, source_total_pressure))`. Candidates pass an explicit rejection cascade (source not overloaded → no pressure → radio-infeasible → target total-guard-margin → target-slice-would-exceed-safe-after-delta → target-total-would-exceed-safe-after-delta → non-positive-score) before splitting negative pressure proportionally by score across surviving candidates — same proportional-sharing philosophy as §5.1, gated by a materially richer feasibility cascade. Rejected candidates can instead produce a small *positive* (retain) bias if the target looks worse than the source by load, total load, or risk.

This function doubles as both the standalone heuristic baseline's upper controller *and* the source of the expert-bias CSV used to shape upper-PPO training (§3.3) — "heuristic baseline" and "imitation/shaping target" are literally the same function evaluated on different rollouts.

5.5 End-to-End Orchestration — `run_heuristic_baseline.py`

`HeuristicUpperPolicy.predict()` recomputes `compute_upper_expert_directional_bias` from live simulator state **every call** (a genuine closed-loop controller, not a pre-recorded trace), then the resulting bias is executed exactly as any other policy's action would be, through the same `GlobalPPO3GNBEnv.step()` path as Section 2's window anatomy:

```
per upper window:
  1. bias  = clip(action, -1, 1)                        # heuristic (or RL) action, identity-mapped
  2. offsets = strong_directional_heuristic_local_executor(bias, ...)   # ONE batched call
  3. apply offsets as live A3 thresholds
  4. freeze per-direction admission quotas from the SAME bias tensor
  5. settle-phase ticks: A3 eval → stability guard → admission controller → handover execution
                         (mid-window: exhausted-quota directions neutralized to 0 dB)
  6. begin measurement window (reward/QoS accounting starts here)
  7. measurement-phase ticks: same per-tick gating
  8. persist offsets as next window's `prev_offsets`
```

Both `run_heuristic_baseline.py` and `run_zero_action_baseline.py` share the identical environment configuration (`build_args` in `run_zero_action_baseline.py`) — the only experimental variable across baseline scripts (and RL evaluation, via the same `evaluate_upper_policy` harness) is the upper-policy's `predict()` implementation, which is the intended apples-to-apples design.

5.6 Zero-Action Baseline

`run_zero_action_baseline.py` — `ZeroActionPolicy.predict()` always returns an all-zero bias vector. Feeding `bias=0` through the executor's deadband (§5.2) forces `raw_offset=0` for every direction, every window — "the no intelligent layer" reference point. A3 handovers can still occur on the unmodified hysteresis-only condition, but no rebalancing pressure is ever injected. Uses the same metrics/CSV pipeline as the heuristic baseline (§5.5), writing to `results/zero_action_baseline/`.

6. Cross-Cutting Notes & Known Discrepancies

Consolidated list of places where the current code diverges from prior design/summary documents, or contains dead/unused paths worth being aware of when interpreting results or extending the system.

6.1 Corrections to `PROJECT_SUMMARY.md`

Claim in `PROJECT_SUMMARY.md`	Current code reality
`raw_offset = bias*4.0` (negative) / `bias*6.0` (positive)	Symmetric `raw_offset = 6.0*bias` for both signs (§5.2)
`target_has_headroom` hard veto zeroing `proto_offset` when unsafe	Not present — `target_load` / `l_safe` only feed an unused debug scalar; no veto anywhere in `strong_directional_heuristic_local_executor` (confirmed by dedicated tests, §5.2)
Over-migration prevented by this load-based veto	Prevented structurally by per-direction admission quotas (§2.5.1), hard handover-count caps, the pingpong/episode stability guard, and mid-window offset neutralization once a quota is exhausted — not by a veto inside offset selection

6.2 Corrections to `hrl_v15_directional_admission.pdf`

The admission-capacity formula `A_ijs = ceil(P_off · min(E,H) · K)` is implemented without its `min(E,H)` load-sensitivity term — see §4.5. The per-block RL observation used by the code (9 features incl. UE counts and a global load/SLA suffix) differs from the PDF's proposed 10-feature block (which specifies raw slice load and an explicit ΔRSRP/target-headroom term) — see §4.2. The lower-agent reward implemented (§4.4) uses different terms and weights than the PDF's "practical starting reward" (§8).

6.3 Dead / unused code worth flagging

- `GlobalPPO3GNBEnv._global_action_penalty()` — defined, never called (superseded by `_bias_smoothness_penalty` + `_negative_bias_magnitude_penalty`).
- `MultiGNBWrapper.get_global_agent_observation()` — a legacy/alternate observation builder not used by `GlobalPPO3GNBEnv._get_observation`.
- `GlobalPPO3GNBEnv.lower_agents = {}` — declared, iterated in `reset()`, but never populated; the upper-PPO training/eval loop never actually exercises the TD3 lower policy (§4.7).
- `DirectionalAdmissionBudgetController`'s `max_target_sla_severity` constructor kwarg — silently swallowed by `**_ignored_legacy_kwargs`, has no effect.
- `MultiGNBWrapper.max_prbs_per_ue` — stored, never referenced again anywhere in the scheduler/admission path.
- `strong_heuristic_local_executor` (non-directional variant) and `coordinated_neighbor_offsets` — both retained for tests/internal reference use only, not on the live evaluation path (§5, table).
- `kbrl_control.py` / `create_kbrl_agent` — dormant, belongs to a retired PRB-budget problem formulation, not the mobility/A3 domain (§5.3).
- Several CLI-exposed upper-reward flags (`--global-neutral-bias-weight`, `--wrong-bias-penalty-weight`, `--sla-reward-weight`, etc.) compute values that are logged but hard-set to 0.0 / excluded from `paper_cost_reward` by design — `scripts/plot_latest_upper_training.py` still plots several of these columns, which will be flat/absent on a current run.

6.4 Test-suite drift

As of this writing, several tests fail against the current codebase because they encode an earlier reward/scenario design: 6 of 17 in `tests/test_upper_reward_causality.py` (assert an older component-sum reward formula that predates the single `paper_cost_reward` term) and 3 of 14 in `tests/test_upper_scenario_variety.py` (assume an earlier, smaller scenario catalog). The formulas in this report were taken from the code paths that actually execute and cross-checked only against the tests that currently pass.